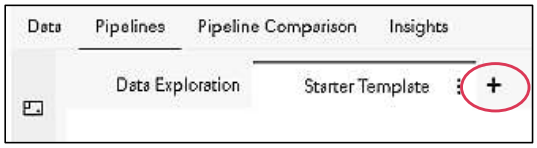
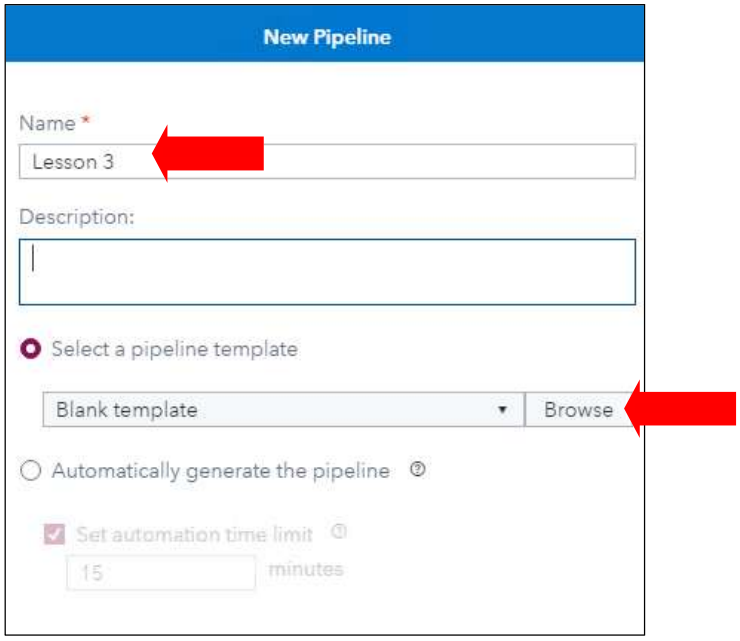
Building a Decision Tree Model with Default Settings

In this demonstration, you use the CPML Demo Pipeline as a starting place in a new pipeline in the **Demo** project. You add a Decision Tree node and build a decision tree model using the default settings of the node.

1. Return to the **Demo** project and click the **Pipelines** tab. Click the plus sign (**+**) next to the Starter Template tab to add a new pipeline.

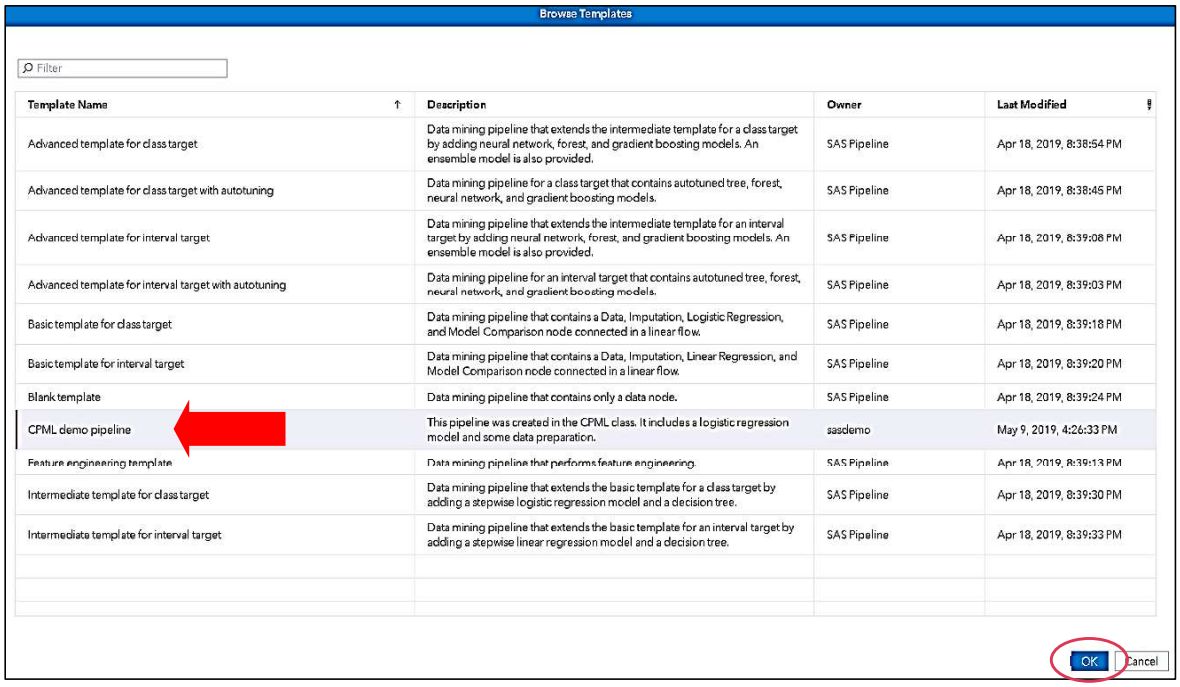


2. In the New Pipeline window, enter **Lesson 3** in the **Name** field. For **Template**, click **Browse**.

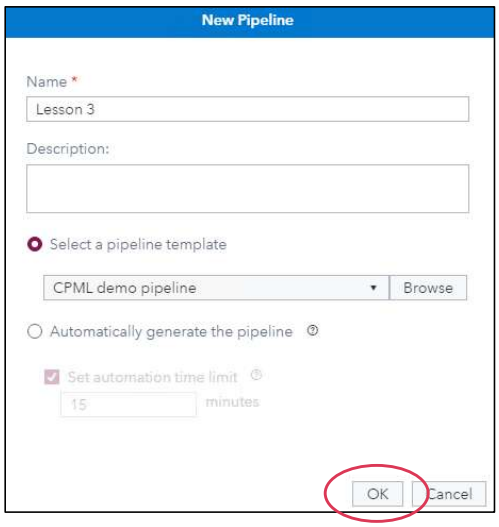


Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

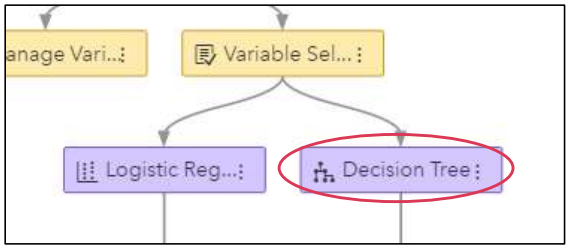
3. Scroll down as needed in the Browse Template window and select **CPML demo pipeline**. This is the pipeline that was saved to the Exchange in the previous lesson. Click **OK**.



4. Click **OK** in the New Pipeline window.



5. In the Lesson 3 pipeline, right-click the **Variable Selection** node and select **Add child node** ⇨ **Supervised Learning** ⇨ **Decision Tree**.

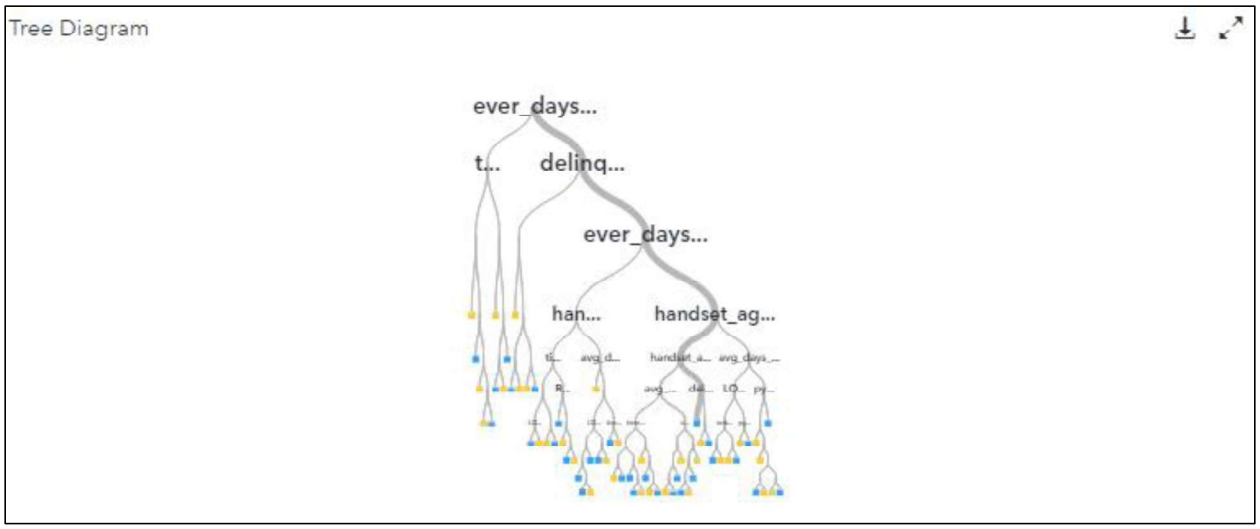


6. Keep all properties for the decision tree at their defaults. Run the **Decision Tree** node.

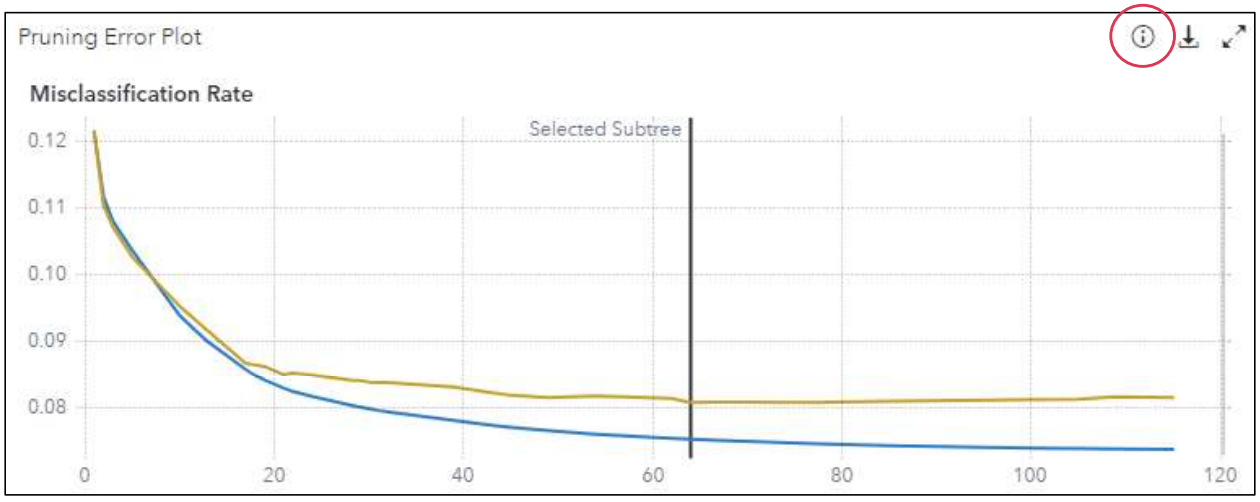
Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

7. Open the results for the Decision Tree node.

   There are several charts and plots to help you in evaluating the model's performance. The first plot is the *Tree Diagram*, which presents the final tree structure for this particular model, such as the depth of the tree and all end leaves.



   The *Pruning Error plot* shows the model's performance based on the misclassification rate – because the target is binary – throughout the recursive splitting process when new end leaves have been added to the final model. Click ⓘ to view an automated description of the pruning error plot.



Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

The *Variable Importance table* shows the input variables that are most significant to the final model. The most important input variable has its relative importance as 1 and all others are measured based on the most important input.

Variable Importance

Variable Label	Role	Variable Name	Validation Impo...	Importance Sta...
Total Days Over Plan	INPUT	ever_days_over_plan	439.6575	0
Handset Age Group	INPUT	handset_age_grp	228.6591	0
Days Suspended Last 6M	INPUT	avg_days_susp	215.8883	0
Delinquent Indicator	INPUT	delinq_indicator	134.5005	0
Total Late Payments Lifetime	INPUT	pymts_late_ltd	85.8653	0

The Validation Importance column is the maximum RSS-based variable importance.

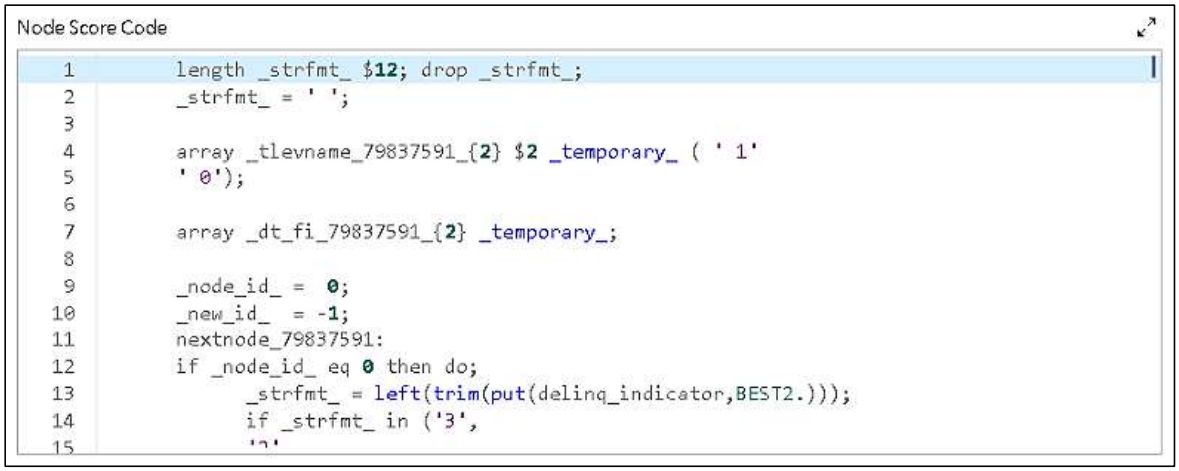
The Importance Standard Deviation column shows the dispersion of the importance taken over partially independent several trees. That's the reason it has all zero values for a single tree. The numbers would be non-zero for forest and gradient boosting. Therefore, you can conveniently ignore this column in a decision tree model.

The Relative Importance column (not shown in the display capture above) measures variable importance based on the change of residual sum of squares (RSS) when a split is found at a node. It is a number between 0 and 1, which is calculated as the RSS-based importance of this variable divided by the maximum RSS-based importance among all the variables (for example, 439.658/439.658=1, 228.659/439.658.128=0.520, and so on). The RSS and relative importance are calculated from the validation data. If no validation data exist, they are calculated instead from the training data.

The Count column (not shown in the display capture above) shows how many times a variable is used in splitting in the decision tree.

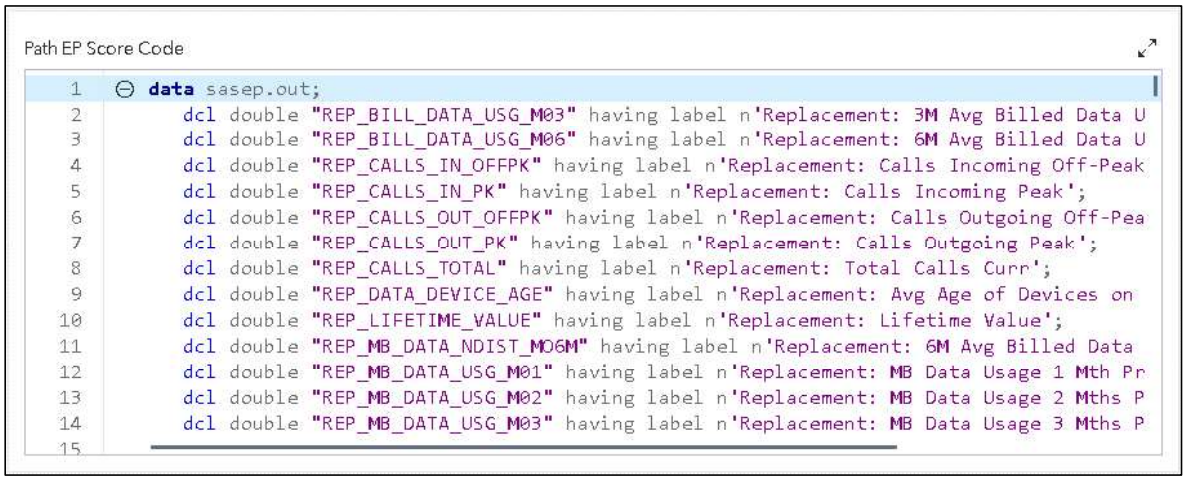
Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Model Studio gives three types of scoring code. The *Node Score Code window* shows the individual score code for a specific node that can be deployed in production. You get this score code against every node in the Data Mining Preprocessing and the Supervised Learning groups that creates DATA step score code. The nodes that create ASTORE score file does not generate this output.



After you add a model in the pipeline (a Supervised Learning node), that node generates two additional score codes and a train code.

The *Path EP Score Code* is the "flow score code," which includes score code for all the nodes until and including that modeling node.



Typically, the Path Score Code or Path EP Score Code is the score code that would be used in other SAS environments (for example, SAS Studio) for scoring. EP stands for Embedded Process, and perhaps it is the underlying engine for in-database scoring (via scoring accelerators). The Path EP Score code that you get from any of the nodes in the process flow creates an ASTORE score file (here due to Text Mining node). Otherwise, you get Path Score Code. In addition, the *DS2 Package Code* (shown below) is score code packaged slightly differently. As a DS2 package, this score code can be used for MAS (SAS Micro Analytic Service). Note that both the Path EP Score Code and the DS2 Package Code are score codes written in DS2.

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

DS2 Package Code

```
package MS_0cd884cf78784110be4361eef84ecfa3_31MAY2019022342719 / overwrite=yes;
   dcl double "REP_BILL_DATA_USG_M03" having label n'Replacement: 3M Avg Billed Data U
   dcl double "REP_BILL_DATA_USG_M06" having label n'Replacement: 6M Avg Billed Data U
   dcl double "REP_CALLS_IN_OFFPK" having label n'Replacement: Calls Incoming Off-Peak
   dcl double "REP_CALLS_IN_PK" having label n'Replacement: Calls Incoming Peak';
   dcl double "REP_CALLS_OUT_OFFPK" having label n'Replacement: Calls Outgoing Off-Pea
   dcl double "REP_CALLS_OUT_PK" having label n'Replacement: Calls Outgoing Peak';
   dcl double "REP_CALLS_TOTAL" having label n'Replacement: Total Calls Curr';
   dcl double "REP_DATA_DEVICE_AGE" having label n'Replacement: Avg Age of Devices on
   dcl double "REP_LIFETIME_VALUE" having label n'Replacement: Lifetime Value';
   dcl double "REP_MB_DATA_NDIST_M06M" having label n'Replacement: 6M Avg Billed Data
   dcl double "REP_MB_DATA_USG_M01" having label n'Replacement: MB Data Usage 1 Mth Pr
   dcl double "REP_MB_DATA_USG_M02" having label n'Replacement: MB Data Usage 2 Mths P
   dcl double "REP_MB_DATA_USG_M03" having label n'Replacement: MB Data Usage 3 Mths P
```

On the other hand, the *Training Code window* shows the SAS training code that can be used to train the model based on different data sets or in different platforms. For example, when you scroll down in the Training Code window, you can see that the TREESPLIT procedure is used to train the decision tree model.

Training Code

```
*------------------------------------------------------------*;
* Macro Variables for input, output data and files;
  %let dm_datalib =;
  %let dm_lib     = WORK;
  %let dm_folder  = %sysfunc(pathname(work));
*------------------------------------------------------------*;
*------------------------------------------------------------*;
  * Training for tree;
*------------------------------------------------------------*;
*------------------------------------------------------------*;
  * Initializing Variable Macros;
*------------------------------------------------------------*;
%macro dm_unary_input;
%mend dm_unary_input;
```

Note: There is a new Score Data node in Model Studio that can be used to collect/accumulate score code at any point in the pipeline.

Finally, the Output window shows the final decision tree model parameters, the Variable Importance table, and the pruning iterations.

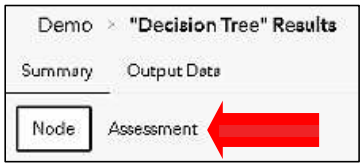
The SAS System

The TREESPLIT Procedure

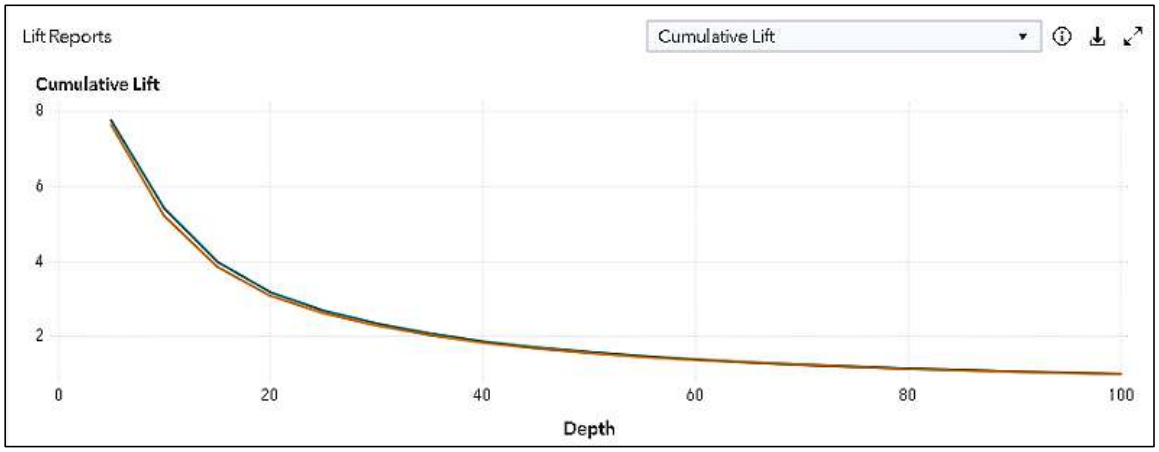
Model Information	
Split Criterion	IGR
Pruning Method	Cost Complexity
Max Branches per Node	2
Max Tree Depth	10
Tree Depth Before Pruning	10
Tree Depth After Pruning	10
Number of Leaves Before Pruning	160
Number of Leaves After Pruning	64

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

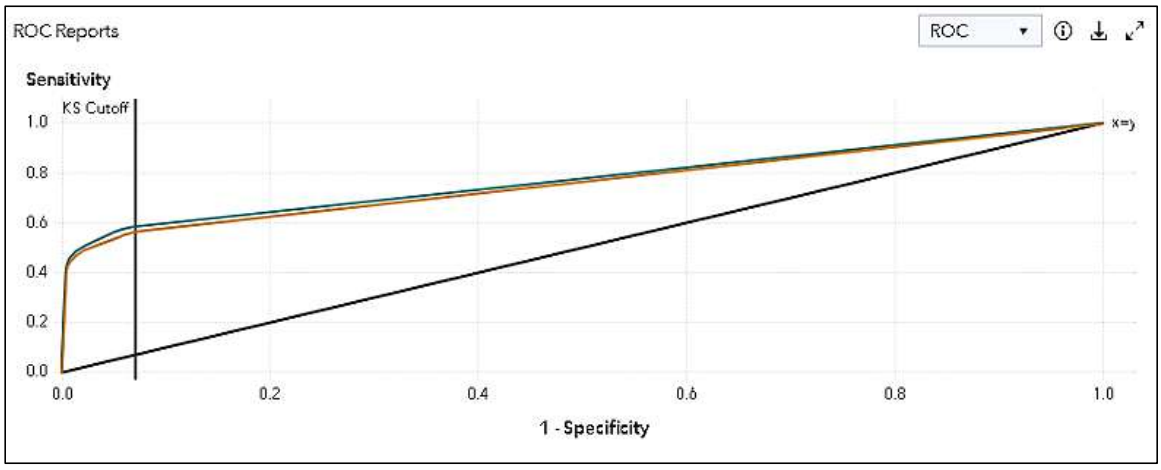
8. Click the **Assessment** tab.



The first chart is the *Cumulative Lift*, showing the model's performance ordered by the percentage of the population. This chart is very useful for selecting the model based on a particular target of the customer base. It shows how much better the model is than the random events.



For a binary target, you also have the *ROC curve*, which shows the model's performance considering the true positive rate and the false positive rate. It is good to foresee the performance on a specific business events, when all positive cases are selected. It shows the model's performance based on the positive cases were predicted right and the positive cases were predicted wrong. ROC is very useful for deployment.

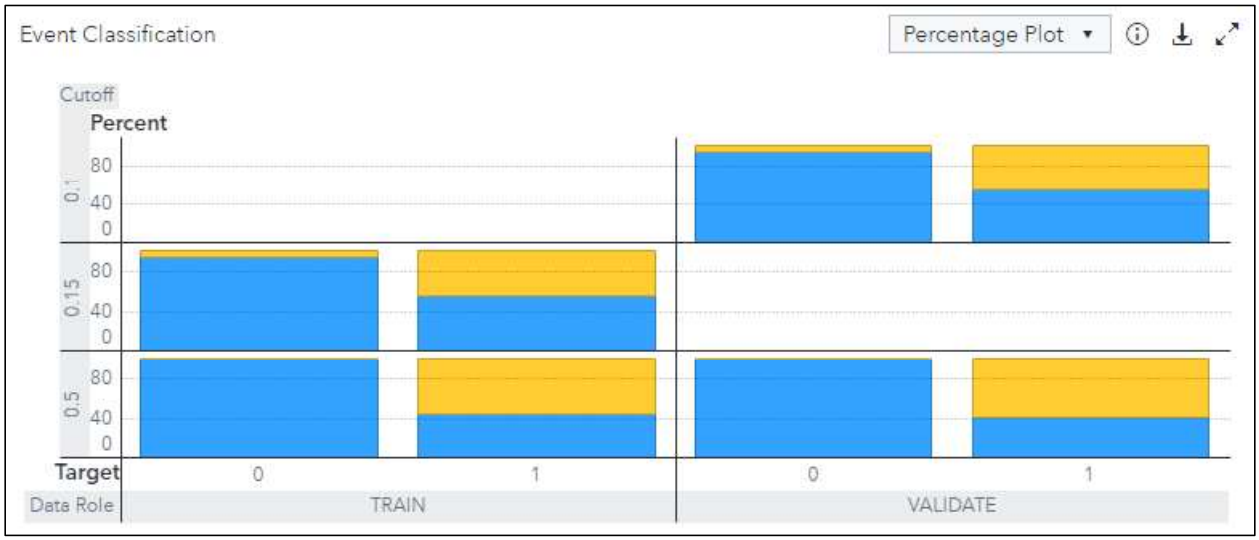


Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

The *Fit Statistics output* shows the model's performance based on several assessment measures, such as average squared error. Expand the table to show that the model has an average squared error of 0.0694 on the VALIDATE partition.

Target ...	Data Role	Partitio...	Formatt...	Sum of ...	Averag...
churn	TRAIN	1	1	39,590	0.0660
churn	VALIDATE	0	0	16,967	0.0694

Finally, you have the *Event Classification output*, which shows the confusion matrix at various cutoff values for each partition. The default view is based on percentages. A count plot and a table of values are also available.



9. Close the Results window.

End of Demonstration

Copyright © 2020, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.